

morphdom	Download count all time build unknown	 npm install morphdom version 2.7.8 updated 1 month ago	Lightweight module for morphing an existing DOM node tree to match a target DOM node tree. It's fast and works with the real DOM—no virtual DOM needed! This module was created to solve the problem of updating the DOM in response to a UI component or page being rerendered. One way to update the DOM is to simply toss away the existing DOM tree and replace it with a new DOM tree (e.g., <code>myContainer.innerHTML = newHTML</code>). While replacing an existing DOM tree with an
1 8	2 7 with any templating language that produces an HTML string. The transformation is done in a single pass of both the original DOM tree and the target DOM tree and is designed to minimize changes to the DOM while still ensuring that the morphed DOM exactly matches the target DOM. In addition, the algorithm used by this module will automatically match up elements that have corresponding IDs and that are found in both the original and target DOM tree.	3 9 DOM node tree and it will efficiently transform the existing DOM node tree to exactly match the target DOM node tree with the minimum amount of changes. morphdom does not rely on any virtual DOM abstractions. Because morphdom is using the <i>real</i> DOM, the DOM that the web browser is maintaining will always be the source of truth. Even if you have code that manually manipulates the DOM things will still work as expected. In addition, morphdom can be used	4 5 entirely new DOM tree will actually be very fast, it comes with a cost. The cost is that all of the internal state associated with the existing DOM nodes (scroll positions, input caret positions, CSS transition states, etc.) will be lost. Instead of replacing the existing DOM tree with a new DOM tree we want to <i>transform</i> the existing DOM tree to match the new DOM tree while minimizing the number of changes to the existing DOM tree. This is exactly what the morphdom module does! Give it an existing DOM node tree and a target
Usage First install the module into your project: <pre>npm install morphdom --save</pre> <i>NOTE: Published npm packages:</i> - <code>dist/morphdom-umd.js</code> - <code>dist/morphdom-esm.js</code> The code below shows how to morph one element to another element.	<pre>var morphdom = require('morphdom'); var el1 = document.createElement('div'); el1.className = 'foo'; var el2 = document.createElement('div'); el2.className = 'bar'; morphdom(el1, el2);</pre>	<pre>expect(el1.className).toEqual('bar'); You can also pass in an HTML string for the second argument: var morphdom = require('morphdom'); var el1 = document.createElement('div'); el1.className = 'foo'; el1.innerHTML = 'Hello John';</pre>	<pre>morphdom(el1, 'Hello Frank'); expect(el1.className).toEqual('bar'); expect(el1.innerHTML).toEqual('Hello Frank');</pre> NOTE: This module will modify both the original and target DOM node tree during the transformation. It is assumed that the target DOM node tree will be discarded after the original DOM node tree is morphed.
9 16 • <code>toNode(Node string)</code> - The node that the <code>fromNode</code> should be morphed to (or an HTML string) • <code>options (Object)</code> - See below for supported options The returned value will typically be the <code>fromNode</code>. However, in situations where the <code>fromNode</code> is not compatible with the <code>toNode</code> (either different node type or different tag name) then a different DOM node will be returned.	10 15 • <code>fromNode (Node)</code> - The node to morph arguments: The <code>morphdom(fromNode, toNode, options)</code> function supports the following morphdom(fromNode, toNode, options) : Node	11 14 • IE9+ and any modern browser • Proper namespace support added in v1.4.0	12 13 Examples See: ./examples/

Supported options (all optional): getNodeKey (Function(node)) - Called to get the Node's unique identifier. This is used by morphdom to rearrange elements rather than creating and destroying an element that already exists. This defaults to using the Node's id property. (Note that form fields must not have a name corresponding to forms' DOM properties, e.g. id.) addChild (Function(parentNode, childNode)) - Called when adding a new child to a parent. By default,	parentNode.appendChild(childNode) is invoked. Use this callback to customize how a new child is added. onBeforeNodeAdded (Function(node)) - Called before a Node in the to tree is added to the from tree. If this function returns false then the node will not be added. Should return the node to be added. onNodeAdded (Function(node)) - Called after a Node in the to tree has been added to the from tree.	onBeforeElUpdated (Function(fromEl, toEl)) - Called before a HTMLElement in the from tree is updated. If this function returns false then the element will not be updated. If this function returns an instance of HTMLElement, it will be used as the new fromEl tree to proceed with morphing for that branch, otherwise the current fromEl tree is used. onElUpdated (Function(el)) - Called after a HTMLElement in the from tree has been updated.	onBeforeNodeDiscarded (Function(node)) - Called before a Node in the from tree is discarded. If this function returns false then the node will not be discarded. onNodeDiscarded (Function(node)) - Called after a Node in the from tree has been discarded. onBeforeElChildrenUpdated (Function(fromEl, toEl)) - Called before the children of a HTMLElement in the from tree are updated. If this function returns
17	18	19	20
24 <pre>onBeforeNodeDiscarded: function(node) { return true; }, onNodeDiscarded: function(node) { return true; }, onBeforeElChildrenUpdated: function(fromEl, toEl) { return true; }, childrenOnly: false, }</pre>	23 <pre>onNodeAdded: function(node) { return true; }, onBeforeElUpdated: function(fromEl, toEl) { return true; }, onElUpdated: function(el) { return true; }, },</pre>	22 <pre>var morphdom = require('morphdom'); var morphedNode = morphdom(fromNode, toNode, { getNodeKey: function(node) { return node.id; }, addChild: function(parentNode, childNode) { parentNode.appendChild(childNode); }, onBeforeNodeAdded: function(node) { return node; } });</pre>	21 skipFromChildren (Function(fromEl, toEl)) - Called when indexing a the fromEl tree. False by default. Return true to skip indexing the from tree, which will keep current items in place after patch rather than removing them when not found in the toEl. childrenOnly (Boolean) - If true then only the children of the fromNode and toNode nodes will be morphed (the containing element will be skipped). Defaults to false. skipFromChildren (Function(fromEl, toEl)) - Called when indexing a the fromEl tree. False by default. Return true to skip indexing the from tree, which will keep current items in place after patch rather than removing them when not found in the toEl.
<pre>skipFromChildren: function(fromEl, toEl) { return false; } })();</pre>	FAQ Can I make morphdom blaze through the DOM tree even faster? Yes. <pre>morphdom(fromNode, toNode, { onBeforeElUpdated: function(fromEl, toEl) {</pre>	<pre>// spec - https:// dom.spec.whatwg.org/#concept- node-equals if (fromEl.isEqualNode(toEl)) { return false } return true } })();</pre>	This avoids traversing through the entire subtree when you know they are equal. While we haven't added this to the core lib yet due to very minor concerns, this is an easy way to make DOM diffing speeds on par with virtual DOM. Isn't the DOM slow? UPDATE: As of v2.1.0, morphdom supports both diffing a real DOM tree with another real DOM tree and diffing a real DOM tree with a
25	26	27	28
32 that is a mirror of the real DOM tree. Whenever the view needs to update, a new virtual DOM tree is rendered. The new virtual DOM tree is then "patched" to match the new virtual DOM tree and the new virtual DOM tree is persisted for future diffing. Both morphdom and virtual DOM based solutions update the real DOM with the minimum number of changes. The only	31 - actualize(document) (non-standard, used to upgrade a virtual DOM node to a real DOM node) - hasAttributeNS(namespaceURI, name) - isSameNode(anotherNode) What about the virtual DOM? Libraries such as a React and virtual-dom solve a similar problem using a <i>Virtual DOM</i>. That is, at any given time there will be a <i>real</i> DOM (that the browser rendered) and a lightweight and persistent virtual DOM tree	30 - node.firstChild - node.nextSibling - node.nodeType - node.nodeName - node.nodeValue - node.attributes - node.value - node.selected - node.disabled following properties and methods of a DOM	29 <i>virtual</i> DOM tree. See: docs/virtual-dom.m No, the DOM <i>data structure</i> is not slow. The DOM is a key part of any web browser so it must be fast. Walking a DOM tree and reading the attributes on DOM nodes is <i>not</i> slow. However, if you attempt to read a computed property on a DOM node that requires a relayout of the page then <i>that</i> will be slow. However, morphdom only cares about the

[illegible]

morphdom nanomorph virtual-dom attr-value-empty-string 0.01ms 0.02ms 0.01ms change-tagname 0.01ms 0.00ms 0.01ms change-tagname-ids 0.02ms 0.00ms 0.02ms data-table 0.60ms 1.38ms 0.80ms data-table2 2.72ms 12.42ms 0.84ms equal 0.50ms 1.33ms 0.02ms id-change-tag-name 0.03ms 0.04ms 0.04ms ids-nested 0.08ms 0.02ms 0.01ms ids-nested-2 0.03ms 0.02ms 0.05ms ids-nested-3 0.03ms 0.01ms 0.02ms ids-nested-4 0.04ms 0.04ms 0.03ms ids-nested-5 0.04ms 0.08ms 49	0.06ms ids-nested-6 0.03ms 0.03ms 0.03ms ids-nested-7 0.02ms 0.03ms 0.02ms ids-prepend 0.03ms 0.03ms 0.02ms input-element 0.02ms 0.04ms 0.00ms input-element-disabled 0.01ms 0.02ms 0.01ms input-element-enabled 0.01ms 0.02ms 0.01ms large 2.88ms 12.11ms 0.35ms lengthen 0.03ms 0.15ms 0.04ms one 0.01ms 0.03ms 0.01ms reverse 0.03ms 0.05ms 0.02ms reverse-ids 0.05ms 0.07ms 0.02ms select-element 0.07ms 0.14ms 0.03ms shorten 0.03ms 0.07ms 0.02ms 50	simple 0.02ms 0.05ms 0.02ms simple-ids 0.05ms 0.09ms 0.04ms simple-text-el 0.03ms 0.04ms 0.03ms svg 0.03ms 0.05ms 0.01ms svg-append 0.06ms 0.06ms 0.07ms svg-append-new 0.02ms 0.02ms 0.18ms svg-no-default-namespace 0.05ms 0.09ms 0.04ms svg-xlink 0.01ms 0.05ms 0.00ms tag-to-text 0.00ms 0.00ms 0.01ms text-to-tag 0.00ms 0.00ms 0.01ms text-to-text 0.00ms 0.01ms 0.00ms textarea 0.01ms 0.02ms 0.01ms todomvc 0.55ms 3.05ms 51	0.32ms todomvc2 0.05ms 0.07ms 0.09ms two 0.01ms 0.02ms 0.01ms <i>NOTE: Chrome 72.0.3626.121</i> 52
56	MIT License 55	Contribute Pull Requests welcome. Please submit Github issues for any feature enhancements, bugs or documentation problems. Please make sure tests pass: npm test 54	Maintainers • Patrick Steele-Idem (Twitter: @psteeleIdem) • Scott Newcomer (Twitter: @puekey) 53
57	58	59	60
64	63	62	19